Secondary Indexes in Partitioned Data

local vs global (GSI)

Watch the 3-min video

In one line: a local index lives on each partition, so writes stay local but queries scatter to every partition and gather; a global index is partitioned by the indexed value, so queries hit one partition but writes touch several and are usually applied asynchronously (eventually consistent).

1 The question the partition key can't answer

Your `orders` table is partitioned by **order_id** across **50 partitions**. A lookup by order_id goes straight to one partition. Then the warehouse team asks: *show me every order with status **shipped***.

Status is not the partition key, so which partition do you ask? All fifty? A **secondary index** answers queries on a non-key column, and in a partitioned database there are two ways to build one.

2 Local index: document-partitioned

Each partition keeps **its own index, covering only its own rows**. Writes are simple: a new order updates the index on the same partition, in the same operation, so the index is always consistent with its data. MongoDB, Cassandra and Elasticsearch use local secondary indexes.

But reads become **scatter-gather**. A query by status must go to **every partition**, then merge the results. The response is only as fast as the **slowest partition**, so tail latency grows with the partition count. Sorting and LIMIT need a merge step too: for "top 20 newest shipped", each partition returns its top 20 and the coordinator merges 50 × 20 rows.

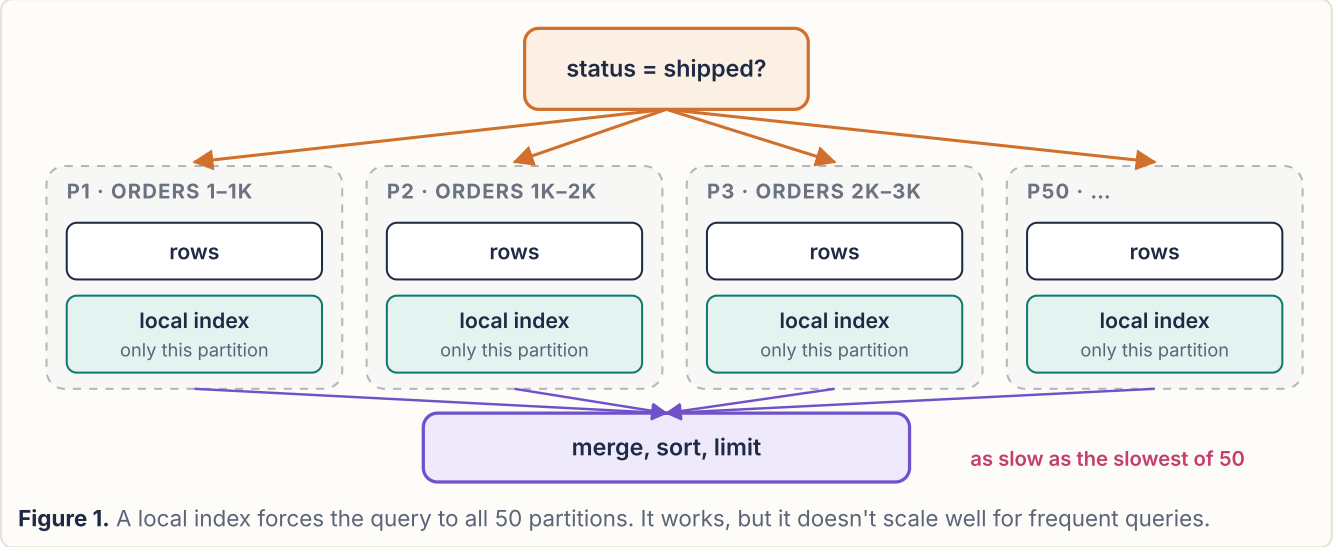
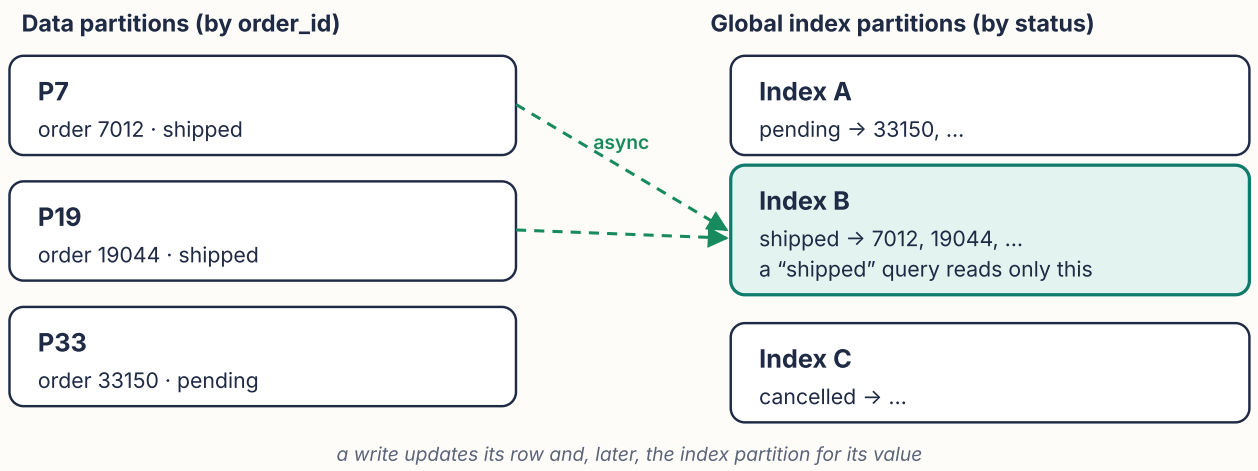


Figure 1. A local index forces the query to all 50 partitions. It works, but it doesn't scale well for frequent queries.

3 Global index: term-partitioned

A global index is **partitioned by the indexed value itself**. All index entries for `shipped` live on one index partition, so a read goes to **just that partition**.

The cost moves to writes. The order row and its index entry live on different partitions, and one update (status pending → shipped) may touch **several**: the data partition, the old value's index partition and the new one's. Doing that atomically would need a distributed transaction, so global indexes are usually **updated asynchronously** and are **eventually consistent**: right after a write, the index may not show it yet.



a write updates its row and, later, the index partition for its value

Figure 2. Orders 7012 and 19044 sit on different data partitions, but both index entries live together on the “shipped” index partition.

4 DynamoDB offers both

	Local secondary index (LSI)	Global secondary index (GSI)
Partition key	same as the table; different sort key	any attribute
Created	only when the table is created	any time
Read consistency	strongly consistent reads supported	eventually consistent only
Updated	with the item, same partition	asynchronously
Capacity	shares the table's	its own capacity

A GSI with **status** as partition key and **created_at** as sort key turns “shipped orders, newest first” into a query on one index partition:

```
GSI  status-created_at:  PK = status, SK = created_at

Query(IndexName='status-created_at',
      KeyConditionExpression='#st = :s',
      ExpressionAttributeNames={'#st': 'status'},   # 'status' is a reserved word
      ExpressionAttributeValues={':s': 'shipped'},
      ScanIndexForward=False,      # newest first
      Limit=20)
```

5 The low-cardinality trap

Status has only a few values. If most of your orders are `shipped`, one index partition takes most index writes and reads: a **hot partition**, like the celebrity key in the last lesson. Two fixes:

- **Bucket suffix:** write the index key as `shipped#3`, with the bucket from `hash(order_id) % 10`. Load spreads over 10 index partitions; a read queries all 10 buckets and merges.

- **Sparse index:** index only the items that have the attribute, e.g. only **pending** orders carry a `pending_since` field. The index stays small, and items drop out once shipped.

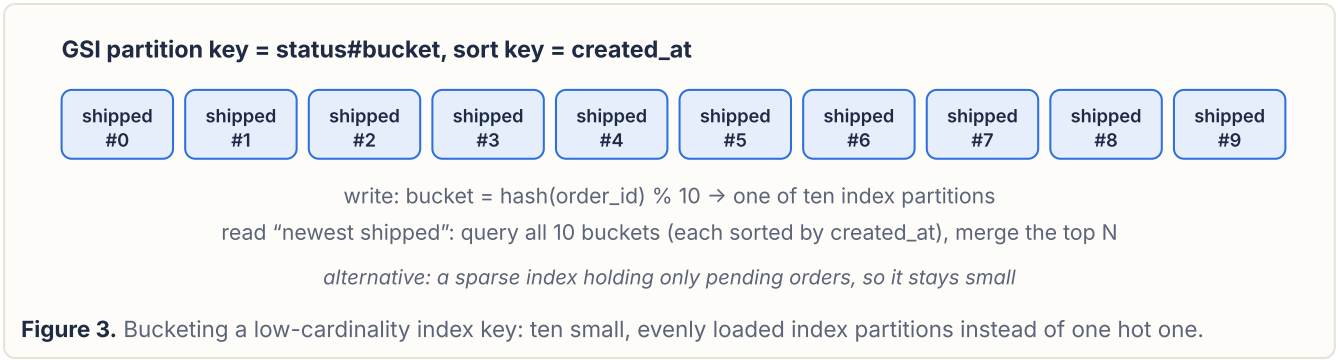


Figure 3. Bucketing a low-cardinality index key: ten small, evenly loaded index partitions instead of one hot one.

WORKED EXAMPLE: THE WAREHOUSE DASHBOARD

- **Local index:** every dashboard refresh asks 50 partitions. If each answers in 5 ms at the median but 1 in 100 answers in 200 ms, then about $1 - 0.99^{50} \approx$ **40%** of refreshes wait for a 200 ms partition.
- **GSI on status#bucket:** each refresh queries 10 small index partitions and merges the top 20, and status changes appear in the index a moment later. Fine for a dashboard; for "did *my* order ship?", read the order by its key instead.

COMMON MISTAKES

- Running a frequent query on a local index and being surprised that p99 grows as you add partitions.
- Reading a GSI right after a write and expecting to see it (read-your-writes is not guaranteed).
- A global index on a low-cardinality key (status, country, boolean) without buckets: one hot index partition.
- Forgetting that every GSI multiplies write cost: each index is another write per changed item.

6 Key terms

Secondary index: index on a non-partition-key column

Document-partitioned: another name for a local index

LSI / GSI: DynamoDB local / global secondary index

Local index: per partition; covers only its rows

Term-partitioned: another name for a global index

Cardinality: number of distinct values

Global index: partitioned by the indexed value

Scatter-gather: ask every partition, merge results

Sparse index: contains only items that have the attribute

7 Interview questions

Q1. Explain local versus global secondary indexes. BASIC

A local index lives on each partition and covers only that partition's rows: writes stay local and consistent, but queries scatter to all partitions and gather. A global index is partitioned by the indexed value: queries hit one partition, but a write may update several partitions, usually asynchronously, so it is eventually consistent.

Q2. Why does scatter-gather hurt tail latency? BASIC

The answer waits for the slowest partition. If each of 50 partitions is slow 1% of the time, about 40% of queries hit at least one slow partition.

Q3. What are the main differences between a DynamoDB LSI and GSI? BASIC

LSI: same partition key, different sort key, defined at table creation, supports strongly consistent reads, shares table capacity. GSI: any partition key, can be added later, eventually consistent, its own capacity.

Q4. Orders partitioned by order_id; the warehouse queries "shipped, newest first" every few seconds. Design it. SCENARIO

A GSI with PK = status (with a bucket suffix, e.g. shipped#0–9) and SK = created_at. Each query reads 10 buckets in descending order and merges the top N. Accept that new statuses appear after a short lag.

Q5. A user updates their email, then immediately logs in by email through a GSI and gets "not found". Why? SCENARIO

The GSI is updated asynchronously, so the new email isn't indexed yet. Fix: retry briefly, look the user up by primary key where possible, or enforce uniqueness with a separate item keyed by email, written in a transaction.

Q6. When is a local index the better choice? SCENARIO

When the query already includes the partition key, e.g. "this customer's orders by date" on a table partitioned by customer_id: it hits one partition and gets consistent reads with cheap writes.

Q7. Why not update a global index synchronously? TRICKY

The row and its index entries live on different partitions, so a synchronous update needs a distributed transaction (e.g. two-phase commit) on every write: slower, and a write fails if any index partition is unavailable.

Q8. Why is a GSI on a boolean like is_active a bad idea? TRICKY

Only two values, so at most two index partitions: if 90% of items are active, one partition takes 90% of index traffic. Add a bucket suffix, or use a sparse index holding only the rare value.

Next: SD-E-026 · Postgres Table Partitioning backendroadmap.com